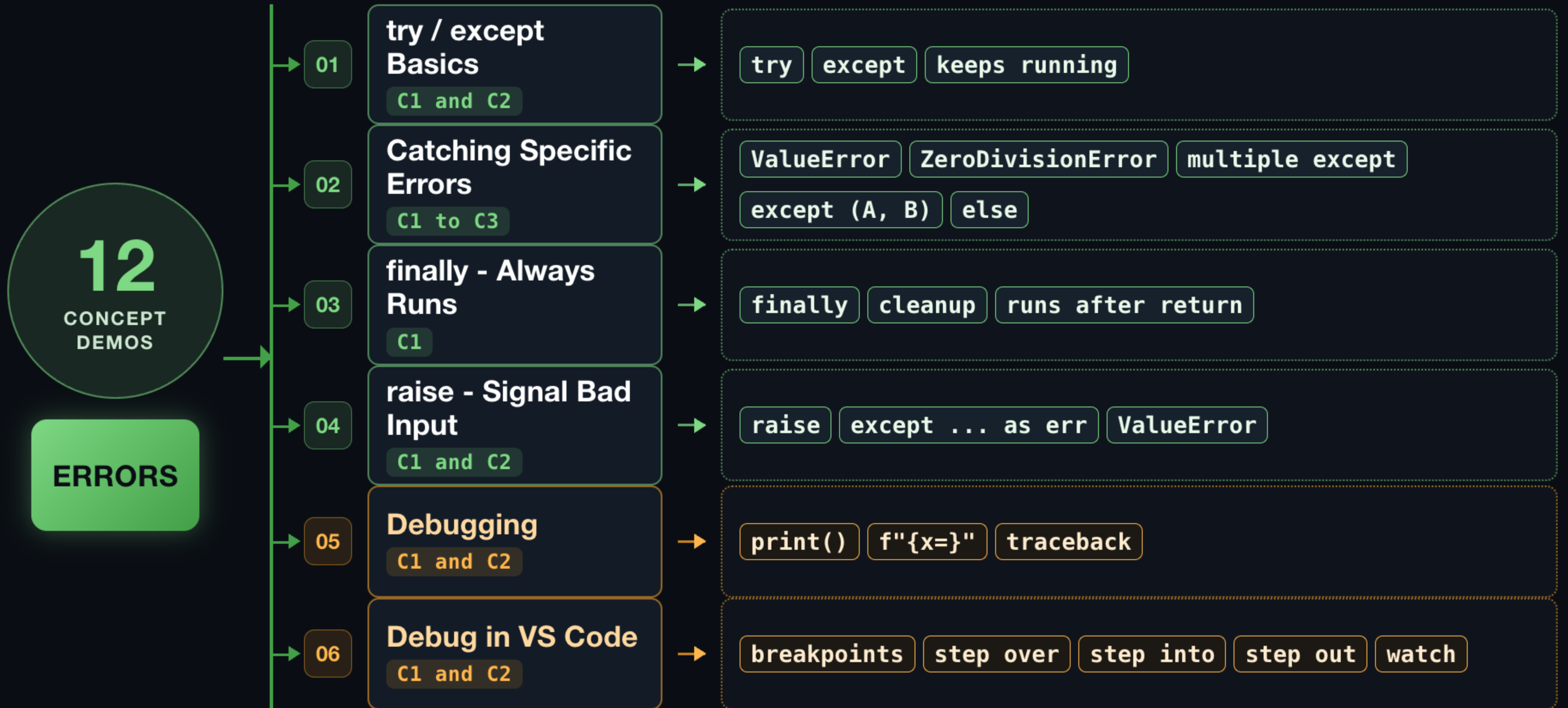


Section 09: Error Handling and Debugging Concept Map

The full path through this section - we open here before each concept, then dive in.



Errors: Keywords Covered

Section 09: Error Handling and Debugging - the keywords in bright green are the new ones we cover here.

Values TrueFalseNone	Classes class	Context Managers with
Logic & Membership andornotinis	Error Handling tryexceptfinallyraise assert	Cleanup & Flow passdel
Conditionals ifelifelse	Modules & Imports importfromas	Generators & Async yieldasyncawait
Loops forwhilebreakcontinue	Scope globalnonlocal	Soft Keywords matchcase_underscoretype
Functions defreturnlambda		

keyword

New in this section

keyword

Covered earlier

keyword

Not covered yet

Errors: **crash**, or **keep going**

try / except is not about the message - it is about what happens NEXT

An unhandled error stops the whole program where it stands. Put the risky lines in **try** and the program **survives the bad value and carries on**.

Without try / except

```
def calculate_books_per_student(b, s):  
    books = int(b)  
    students = int(s)  
    return books / students
```

RUNNING

- ("100", "20") 5.0
- ("abc", "20") **ValueError**
- ("100", "0") never reached
- everything after never reached

One bad value ends the program

With try / except

```
def calculate_books_per_student(b, s):  
    try:  
        books = int(b)  
        students = int(s)  
        return books / students  
    except Exception:  
        return "Something went wrong with the input."
```

RUNNING

- ("100", "20") 5.0
- ("abc", "20") Something went wrong...
- ("100", "0") Something went wrong...
- The program kept running.

Every call answers; nothing stops

try holds the lines that might fail. **except** is what to do instead of crashing. **The lines after the call still run** - that is the whole win.

Errors: The **try** / **except** Block

try / **except** lets your program run risky code and catch any error, so one bad value cannot crash the whole program.

#	CONCEPT	SAMPLE CODE
1	Bad input crashes everything One bad value ends the program; nothing after it runs.	<pre>int("abc") # ValueError</pre>
2	Catch it with try / except try Wrap the risky lines; except runs instead of crashing.	<pre>try: ... except Exception:</pre>

Errors: one **try**, many **exits**

The kind of error decides which **except** block runs

Name the error you expect and each problem gets **its own honest reply** instead of one vague message.

```
try:
    books = int(total_books)
    students = int(total_students)
    result = books / students
except ValueError:
    return "Books and students must be whole numbers."
except ZeroDivisionError:
    return "There must be at least one student."
else:
    # only when NOTHING was raised
    print("Input looked good, sharing the books.")
    return result
```

NO ERROR - else RUNS
("100", "20") -> **Input looked good, sharing the books.**
5.0

except ValueError
("abc", "20") -> **Books and students must be whole numbers.**

except ZeroDivisionError
("100", "0") -> **There must be at least one student.**

When two errors deserve the **SAME** reply, catch them together
except (ValueError, ZeroDivisionError): answers both with one line -
"Please enter valid numbers (students above zero)."

Read it top to bottom: Python takes the **first except that matches**, and **else** is the lane that opens only when no error was raised at all.

Errors: Catching **Specific** Exceptions

A specific **except** block catches one named error type, so each kind of problem gets its own clear message.

#	CONCEPT	SAMPLE CODE
1	Separate except blocks <code>except</code> Each error type gets its own reply.	<code>except ValueError:</code>
2	Success-only step <code>else</code> else runs only when try had no error.	<code>else:</code>
3	Catch two together <code>except (A, B)</code> One message for two error types.	<code>except (ValueError, ZeroDivisionError):</code>

Errors: **finally** always runs

Cleanup you cannot forget

A **finally** block runs whichever way the call ended - success, caught error, **even after a return**. It is where you close things and tidy up.

```
try:
    books = int(total_books)
    students = int(total_students)
    return books / students
except (ValueError, ZeroDivisionError):
    return "Please enter valid numbers..."
finally:
    print("Finished sharing the books.")
```

The good call - ("100", "20")

Finished sharing the books.

5.0

finally printed BEFORE the value came back

The bad call - ("100", "0")

Finished sharing the books.

Please enter valid numbers...

same line, same place, after the error too

try succeeded

finally runs

except handled it

finally runs

return already happened

finally still runs

Look at the order in both outputs: **"Finished sharing the books."** appears **before** the returned value. The return value waits while **finally** does its job.

Errors: **finally**, Cleanup That Always Runs

A **finally** block always runs after the try and except, whether the code worked or failed, so it is where you put cleanup that must never be skipped.

#	CONCEPT	SAMPLE CODE
1	The finally block <code>finally</code> Runs whether the try worked or failed.	<code>finally:</code>

Errors: **raise** one, catch it, read it

Some bad input is not a Python error; it is YOUR rule

Nothing crashes on minus five books; Python is happy. **raise** is how you say **this is not allowed here**, and the sentence you write is the sentence the caller reads.

INSIDE THE FUNCTION, you raise it

```
if students <= 0:  
    raise ValueError("There must be at least one student")
```

THE SAME
SENTENCE

WHERE YOU CALL IT, you catch it

```
try:  
    calculate_books_per_student("100", "0")  
except ValueError as err:  
    print("Caught a problem:", err)
```

WHAT PRINTS

5.0 the good call still returns

Caught a problem: **There must be at least one student**

CONTINUE PROGRAM – The program kept running

raise

Fires only when your rule is broken. Good input returns exactly as it always did.

as err

Hands you the error object, so you can print what went wrong.

your sentence

Is the error screen your user reads. Write one they can act on.

Errors: **raise** to Reject Bad Input

raise stops a function on purpose when the input breaks a rule, and hands the caller a clear error message to catch and read.

#	CONCEPT	SAMPLE CODE
1	Signal an error on purpose raise Stop the function and send a clear message.	<pre>raise ValueError("total books cannot be negative")</pre>
2	Catch it where you call except A raised error is caught like any other.	<pre>except ValueError as err:</pre>

Errors: three kinds of **bug**, and when each one bites

The three bugs are told apart by **WHEN** they hit you

One stops the file before line 1. One stops it halfway. One never stops anything - it just **gives you the wrong answer**, which is why it is the dangerous one.

BEFORE ANYTHING RUNS

MID-RUN

NEVER CRASHES

BEFORE LINE 1

Syntax error

```
def calculate(books, students)
# the ":" is missing
```

SyntaxError – nothing runs, not even line 1

Python reads the **WHOLE** file first. **Fix the spot the caret points at.**

MID-RUN

Runtime error

```
return books / studphts
# typo: students
```

NameError: name 'studphts' is not defined

Earlier lines already ran. **Read the traceback bottom-up** - the last line names the error.

NEVER

Logic bug

```
return books * students
# should be /
```

Buggy result: 2000 (expected 5.0)

No error at all. **Print the values to see them:** debug: books=100 students=20

`print(f"{books=}")` prints the **name and the value** together - the fastest way to see what your code actually has, when nothing has crashed to tell you.

Errors: Debugging, Find and Fix Bugs

Debugging is finding why code misbehaves and fixing it, by reading error messages and printing values to see what really happened.

#	CONCEPT	SAMPLE CODE
1	Logic bug {books=} Wrong answer, no crash. Print the values to spot it.	<pre>print(f"{books=} {students=}")</pre>
2	Runtime error NameError Crashes mid-run. Let it crash, then read the traceback from the bottom up.	<pre>NameError: name 'studphts' is not defined</pre>

Errors: **Debug in VS Code**, Pause and Step Through Your Code

A **breakpoint** pauses your program on a line so you can look at every variable, then walk forward one line at a time instead of guessing.

#	CONCEPT	SAMPLE CODE
1	A call chain Step Into One function calls another, so Step Into goes down and Step Out comes back.	<pre>each = calculate_books_per_student("100", "20")</pre>
2	Breakpoint in a loop Continue It stops once per pass, so Continue walks you through the rows.	<pre>for total_books, total_students in orders:</pre>

Debugging in VS Code: what each toolbar button does

The toolbar appears at the top the moment your program **pauses at a breakpoint**. Left to right, these are all six buttons. Five keys are the same everywhere; only **Restart** differs on Windows.

#	BUTTON	NAME	MAC	WINDOWS	WHAT IT DOES
1		Continue	F5	F5	Runs at full speed to the next breakpoint , or to the end. While running, this button becomes Pause.
2		Step Over	F10	F10	Runs this line, stops on the next. A function on the line runs fully, but you never go inside. The one you press most.
3		Step Into	F11	F11	Goes inside the function called on this line, so the next stop is that function's first line.
4		Step Out	Shift + F11	Shift + F11	Finishes the function you are inside and drops you back on the line that called it.
5		Restart	Shift + Cmd + F5	Ctrl + Shift + F5	The green one. Starts the same session again from line 1, keeping every breakpoint.
6		Stop	Shift + F5	Shift + F5	The red one. Ends the session. Your breakpoints stay set, so F5 picks up again later.

The dot in buttons 2, 3 and 4 is the function call: the arrow hops over it, goes down into it, or comes up out of it. Most bugs need only **Step Over** and the **red square**.